Distrain: Asynchronous Weighted-Contribution Training of Language Models Across Heterogeneous Devices

Frane Bandov
offbyte
frane@offbyte.com

Abstract

Distrain trains language models across heterogeneous volunteer devices without synchronization between nodes. Each device trains locally against a checkpoint, compresses its weight update into a delta, and pushes to a coordinator that merges incoming contributions by weighted averaging. There is no coordination needed, as delta pushes are independent and fast and slow participants can coexist and contribute side by side. Hyperparameters are derived from device capability at startup, so volunteers do not need to configure anything. The system is a single Rust binary that can run on any viable machine.

We have designed and tested the protocol on a 125M transformer and benchmarked it against a single-GPU baseline with similar hyperparameters. On FineWeb-Edu a three-node distributed training reaches a loss of 5.98 against a baseline loss of 4.8. We attribute this gap to three candidates: top-k compression, merge staleness, and weighted averaging over multiple deltas. A consumer-hardware run across devices with a 10× throughput range converges. We report failures on loss-based learning rate scheduling and low-rank delta compression, which have both impacted the protocol design.

1 Introduction

Training large language models normally requires tightly coupled GPU clusters with low-latency interconnects as data-parallel SGD with AllReduce [9], ZeRO [16], and FSDP [28] assume homogeneous hardware, reliable high-bandwidth connections, and synchronous execution. While there is recent work that pushes against these constraints, such as DiLoCo [4], which reduced communication frequency through inner-outer optimization, and OpenDiLoCo [7], which scaled this to 10B parameters across continents, or DeMo [15], which reduced communication bandwidth by 85× via decoupled momentum with DCT-based sparsification, all remain synchronous, at least in the outer optimization steps, which leads to slow or intermittent devices holding up the rest.

Distrain is a distributed training system without node-level synchronization, designed for volunteer hardware, where each device contributes independently and the coordinator merges incoming deltas by staleness-weighted averaging. The protocol replaces outer synchronization with this merge and every training hyperparameter is derived from device capability at startup, since volunteers do not need to tune them. It runs as a single Rust binary on GPUs or CPUs without any external dependencies.

Our primary testing was on a 125M transformer. We established a single-GPU baseline and ran one consumer-hardware curve plus six datacenter curves on FineWeb-Edu. We have found that Distrain reaches a loss of 5.98 while baseline reaches 4.8. We suspect this ∼1.2 loss difference comes from top-k compression, merge staleness, and weighted averaging over multiple deltas. As a consumer-hardware feasibility test we ran the system on three laptops with very different speeds,

the fastest was 10x faster than the slowest, and the training still converged. Further we also report negative results as we have found that loss-based learning rate scheduling and low-rank delta compression do not positively impact the design and performance.

2 Related Work

Distributed training research can be clustered into two groups. The first group around synchronous methods has mostly focused on reducing how often nodes communicate. Data-parallel SGD with AllReduce [9], and the memory-sharding extensions FSDP [28] and ZeRO [16], assume homogeneous hardware and lock-step synchronization. Local SGD [22, 10] and FedAvg [13] relaxed the every-step assumption by letting workers take multiple local steps before syncing and DiLoCo [4] pushed this further with an outer Nesterov optimizer and reached competitive quality with 500× fewer communication rounds. Further, OpenDiLoCo [7] scaled it to 10B parameters across continents, and DeMo [15] reduced bandwidth by 85× through decoupled momentum with DCT-based sparsification. The outer synchronization did not change across this work, meaning nodes still have to agree on when a round ends.

The second group focuses on decentralized methods and has mostly targeted volunteer hardware for inference. Hivemind [18] built the infrastructure for training across the open internet, and SWARM parallelism [19] showed that large models can be trained communication-efficiently over unreliable nodes. However, in practice the decentralized stack has mostly served inference (Petals [2]) rather than pretraining. The failure mode decentralized work handles well, namely node churn, is not what the synchronous work optimizes against, and the failure mode the synchronous work handles well, infrequent but reliable communication between cooperating nodes, is not what volunteer hardware offers.

Volunteer pretraining sits in the middle, with nodes that are genuinely heterogeneous, residential uplinks slower than datacenter connections, and participants dropping in and out. Asynchronous SGD [17] drops sync entirely by letting workers apply gradient updates to shared memory without locking; DistBelief's Downpour SGD [3] extended the idea across parameter-server clusters. Convergence holds under bounded staleness [5]. Both operate on per-step gradients, which on residential uplinks removes the amortization DiLoCo's round structure provided. Distrain attempts to merge the best of both worlds: it sends weight deltas rather than gradients, removes the outer sync, and uses top-k sparsification [11] with error feedback [23, 8] to keep push sizes small enough for residential connections. Low-rank compression [6] works well for fine-tuning because fine-tuning updates are structurally low-rank; pretraining deltas are not, and low-rank compression failed in our experiments (§5.4).

3 Method

Distrain is a coordinator with an arbitrary number of nodes pushing weight deltas to it. The coordinator merges pushed deltas into checkpoints. The rest of this section describes the merge protocol, the compression applied to each delta, and the zero-configuration calibration that lets a volunteer start training without tuning hyperparameters.

3.1 Weighted Contribution Protocol

Each node trains against the most recent checkpoint it has pulled. After h_i local AdamW steps, node i has moved its local parameters from θ_{v_i} to $\theta_{v_i}^{(i)}$. It sends the difference,

$$\delta_i = \theta_{v_i}^{(i)} - \theta_{v_i}, \tag{1}$$

not the gradients. The delta carries the accumulated effect of h_i optimizer steps, including momentum, adaptive per-parameter learning rates, and weight decay. DiLoCo sends a similar quantity as a "pseudo-gradient" to its outer optimizer [4]. The alternative, transmitting compressed momentum every step as in DeMo [15], preserves more signal per communication but requires communication every step, which residential uplinks cannot sustain.

The coordinator assigns each incoming delta a weight

$$w_i = T_i \cdot \gamma^{s_i}, \tag{2}$$

where T_i is the tokens node i processed in this round, $s_i = v_{\text{current}} - v_i$ is its staleness against the current checkpoint, and $\gamma = 0.9$ is a decay factor. Deltas with $s_i > 10$ are discarded. Weighting by tokens rather than steps handles batch-size differences between nodes. The decay factor and cutoff were chosen by hand during early runs and have not been ablated; at $s = 10$ the weight is $\gamma^{10} \approx 0.35$, which we treated as the practical floor.

Incoming deltas are accumulated into a bucket. When enough have arrived (see below), the bucket is merged by weighted averaging and the result is applied to produce the next checkpoint:

$$\bar{\delta} = \frac{\sum_i w_i \,\text{decompress}(\delta_i)}{\sum_i w_i}, \qquad \theta_{v+1} = \theta_v + \bar{\delta}. \tag{3}$$

The bucket resets at each checkpoint. Merging is commutative and associative inside a bucket, so the order of arrivals does not affect the merged checkpoint, and deduplication by (node id, sequence number) prevents a delta from being applied twice if the coordinator receives a retry.

Our experiments found that treating $\bar{\delta}$ as the full outer update ($\eta_{\text{outer}} = 1.0$, no outer momentum) outperformed adaptive outer learning rates on the order of $[0.07, 0.97]$ that discarded most of the merged delta's magnitude. The supporting data is in §5.

Checkpoint production. A new checkpoint is produced when the bucket has received deltas from at least half of the active nodes, where a node is considered active if it has pushed within the last three checkpoint cycles. Both thresholds are derived from the observed contributor set, not configured. The hard minimum is two contributions, set after an early run in which a fast node produced checkpoints alone while other nodes were still training and the concurrent work on the slower nodes was abandoned. The coordinator holds the bucket in memory and serves as the single writer to object storage. When the threshold is met, it freezes the current contributions, downloads the referenced deltas from object storage, computes the weighted average, uploads the new checkpoint, and resets the bucket. The bucket is periodically snapshotted to object storage and always flushed immediately after a checkpoint, so a coordinator restart loses at most the contributions received since the last snapshot. This design assumes a single coordinator; running multiple coordinators against the same bucket would require compare-and-swap on the bucket writes and is left for future work.

3.2 Delta Compression

Raw model deltas are large. A 7B-parameter model produces roughly 14 GB per delta in BF16, which residential uplinks cannot carry in any reasonable time. We compress each delta in three stages before the node pushes it.

The first stage is top-k sparsification. The node keeps the k% of delta values with the largest magnitude and drops the rest. The value of k depends on the node's measured upload bandwidth: a datacenter connection ($\gtrsim$50 MB/s) retains 90%, fast residential keeps 20%, and slow residential drops to 1–2%. Within a single node, k also rises over training, from around 0.5% early on, when the largest updates dominate, to around 7% later, when gradient mass spreads across more parameters. The bandwidth-driven k and the training-phase drift interact: the bandwidth setting is a floor, and the training-phase adjustment nudges k upward from that floor when the delta's distribution flattens out.

The second stage is error feedback. Values dropped by sparsification stay in a local buffer e_i and are added to the next round's delta before sparsification runs again. Parameters whose updates were small in one round but grew in the next enter the top-k set when the buffered mass tips them over the threshold. Error feedback has strong theoretical support within a single accumulation window [8]. The guarantee does not carry across checkpoint production: when the coordinator produces a new checkpoint, each node clears its buffer, and any mass still sitting in e_i is lost. We have not measured how much signal this discards; the current design treats the loss as the cost of the async protocol and accepts it.

The third stage is index encoding followed by zstd. Sparse indices are sorted and delta-encoded, and the entire payload is compressed. At 90% retention a 125M delta is around 300 MB after compression; at 1–2% retention it is around 5 MB.

3.3 Device Calibration

Volunteer nodes arrive with unknown hardware, unknown bandwidth, and volunteers who have not read the paper. The node has to start training without being asked anything. Four hyperparameters are derived from the device at startup rather than configured.

The compute backend is probed in order: CUDA if available, then Metal or Vulkan via wgpu, then CPU as a fallback. A node with a GPU that is too small for the model will fall through to CPU.

Batch size is a function of available VRAM and the model's hidden dimensions, layer count, FFN width, vocabulary, and sequence length. If the first estimate triggers an out-of-memory error, the node halves the batch and retries. The estimate is a VRAM fit, not a quality choice: on an A40 with 48 GB the fit gives batch 26 for the 125M model, but our experiments (§5.3) show that smaller batches yield better per-token convergence for this protocol. All results we report use batch 4, which is the setting the node uses when VRAM is not the constraint.

Inner steps h_i target a push interval of about 60 seconds, so that upload overhead stays a small fraction of each round:

$$h_i = \mathrm{clamp}\left(\left\lfloor \frac{T_{\mathrm{target}}}{t_{\mathrm{step}}} \right\rfloor,\ h_{\min},\ h_{\max}\right), \tag{4}$$

where t_{step} is the measured wall time per step and the clamp bounds $h_{\min} = 50$, $h_{\max} = 500$ were set during development. After the first round, the node refines h_i from its observed upload throughput.

Compression retention is picked from measured upload bandwidth with the aim of keeping upload time under 10 seconds. A short bandwidth probe at the first push seeds the estimate;

subsequent pushes refine it. The inner learning rate is not calibrated per device. We use a constant 3×10^{-4} across nodes, a value that held up across the baseline and all reported distributed runs.

3.4 Shard Assignment

Training data is split into fixed-size shards of 10 million tokens, small enough that a node can download one quickly and large enough that it will take many minutes to work through. Each node decides which shards to train on by hashing its own identifier together with the current checkpoint version; the hash seeds a pseudo-random selection over the shard list. The coordinator is not involved and never computes or stores any node's assignment. The assignment is deterministic: anyone who knows the node id and the checkpoint version can compute the same shards, which also means a node that crashes and restarts against the same checkpoint resumes on the data it was already working through. When the checkpoint advances, the version input to the hash changes and each node picks a new shard set, so a node that keeps pushing against the same checkpoint stays on the same shards, and the expected coverage across all nodes is close to uniform over the shard set without any explicit coordination.

4 System Implementation

The method describes the protocol and this section describes what actually runs. The coordinator and node are two Rust processes that exchange metadata over HTTP and delta tensors through object storage.

4.1 Architecture

Distrain has two components, both written in Rust: a coordinator process and a training node. The coordinator is a single process that serves an HTTP API, holds the contribution bucket in memory, and periodically writes its state to object storage. The node is a single binary that trains, compresses, uploads deltas, and pulls checkpoints. Both use the Burn deep learning framework [21], which compiles the same model code to CUDA, Metal and Vulkan (through wgpu), and a CPU backend. Persistent storage is any S3-compatible object store; we use Cloudflare R2 in deployment and MinIO for local development. Checkpoints are written in the safetensors format, and training data is stored as binary uint16 shards.

The delta push flow is split between the node, object storage, and the coordinator. The node uploads the compressed delta bytes directly to object storage under a key that encodes the checkpoint version and an assigned sequence number. It then sends a small metadata record to the coordinator: the node id, checkpoint version, sequence number, delta key, inner step count, training loss, tokens processed, wall time, compressed size, and the dense and sparse norms of the delta. The coordinator never receives or stores delta tensors directly; it adds the metadata to its in-memory bucket and downloads the referenced deltas from object storage only when it is time to aggregate. Browser and WASM nodes that cannot reach object storage directly (because of CORS) proxy their uploads through a coordinator endpoint that forwards the bytes to storage without interpreting them.

Checkpoint distribution works the other way: nodes read the current version number from the coordinator and pull the checkpoint bytes directly from object storage. The coordinator is off the read path for checkpoints, which keeps its workload bounded by the much smaller metadata and aggregation traffic.

Aggregation runs on CPU using Burn's ndarray backend. Coordinators are deployed on small cloud instances that may not have a GPU, and at 125M with a few contributions the weighted

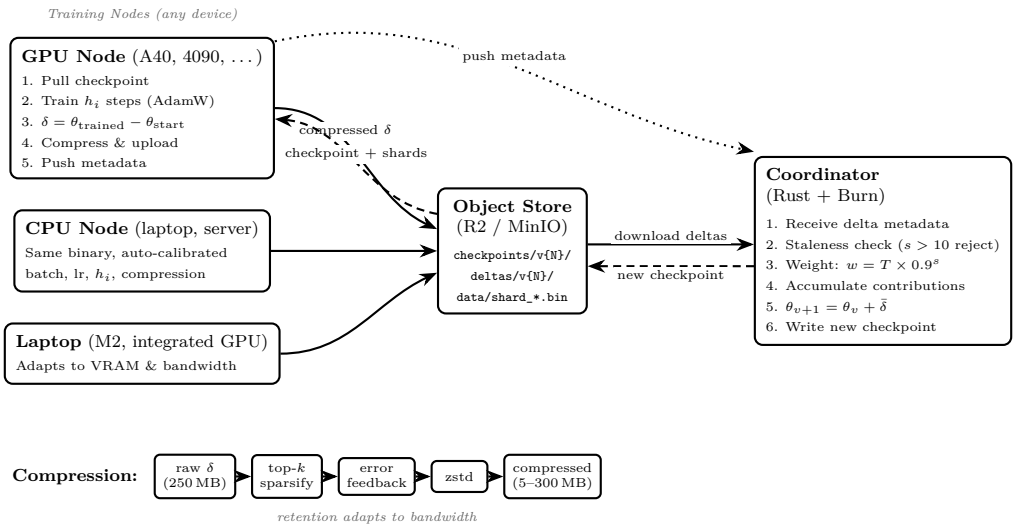

Figure 1: System architecture. Nodes train independently, compress deltas via top-k sparsification with error feedback, and upload to object storage. The coordinator receives push metadata, downloads deltas from storage when aggregating, and writes new checkpoints via weighted averaging of contributions.

average plus outer update finishes in a couple of seconds, small compared to the minutes between checkpoints. Building with a CUDA feature flag switches the aggregation backend to GPU for larger models if the coordinator host has one.

The node runs a loop of pulling the latest checkpoint, training h_i inner steps, computing the delta, compressing it, uploading to storage, and pushing metadata to the coordinator. Compression runs on a separate thread from training, so the GPU is free to start the next round of training while the previous delta is being sparsified and zstd-encoded in the background. The backend (CUDA, Metal, Vulkan, or CPU) is selected at startup by the calibration code described in §3.3, not at compile time.

Shipping the system as Rust binaries was a pragmatic choice: it avoids the Python, CUDA toolkit, and pip environment setup that the systems we cite require. For volunteers who are willing to run a binary but not willing to resolve a broken wheel install, the difference is non-trivial. We do not claim this is a contribution of the protocol itself, only of the implementation.

Figure 1 shows the full system.

4.2 Model

We implement a standard decoder-only transformer [26]: Grouped-Query Attention [1] with RoPE [24] ($\theta = 500{,}000$), SwiGLU feed-forward network [20], RMSNorm [27] (pre-norm), and tied embeddings with learned output scaling, following LLaMA [25]. The optimizer is AdamW [12]. The tokenizer is Mistral v0.3 (vocabulary size 32,768).

We evaluate two model sizes:

- 125M parameters: 768 hidden, 12 layers, 12 attention heads, 4 KV heads.
- 1.13B parameters: 2048 hidden, 24 layers, 16 attention heads, 4 KV heads.

4.3 Training Data

We use FineWeb-Edu [14], tokenized with the Mistral v0.3 tokenizer and split into shards of 10M tokens each (∼20 MB per shard in uint16 binary format). The version we downloaded totals 11.01 billion tokens across 1,102 shards. A held-out set of shards is reserved for validation and is never used in training assignments; validation loss reported in §5 is computed on this set.

5 Experiments

5.1 Setup

All curves train the 125M transformer described in §4.2 on FineWeb-Edu. The single-GPU baseline (Curve A) runs on 1× RTX 4000 Ada with batch 4, learning rate 3×10^{-4}, and a cosine schedule with warmup over the full training run. Curve A is what we benchmark against throughout the rest of this section.

Curve B is the first distributed run, on three laptops: two MacBook Pros with Apple Silicon GPUs via Metal, and a MacBook with an Intel CPU. It reached loss 6.3 over 46.7M tokens and 2928 checkpoints, limited by the 10× throughput gap between the fastest and slowest device and running over multiple days of wall time. The per-device throughput values, measured from per-round node timings, are in Table 1. Subsequent curves moved to datacenter GPUs so that experiments could finish in hours rather than days; Curve B remains the only evidence that the protocol converges on consumer hardware at all.

Curves C through H run on three datacenter GPUs, with the coordinator on a separate smaller pod. Curves C and D used mixed and A4000 RTX hardware during early development; Curves E through H moved to 3× A40 (48 GB) for controlled comparison. The baseline and each distributed curve use the same FineWeb-Edu shards and the same tokenizer; the curves differ in what changed between them, described in §5.2. Curve F, run with batch 25 and linear LR scaling, turned out to be invalid because of a data loader bug and is excluded from the main comparison; we describe it in the negative results subsection (§5.4).

Table 1: Consumer-hardware devices used in Curve B.

Device	Processor	Memory	Backend	Throughput
MacBook Pro 14"	Apple M2 Pro	32 GB unified	GPU (Metal)	∼1,000 tok/s
MacBook Air 13"	Apple M2	24 GB unified	GPU (Metal)	∼800 tok/s
MacBook Pro 13"	Intel i7-1068NG7	32 GB	CPU	∼100 tok/s

5.2 Systematic Optimization (Curves B–H)

We ran seven distributed curves. Some were planned, some were follow-ups and optimizations to previous results and some were new approaches. The results are in Table 2.

Curve B was the consumer-hardware run described in §5.1 and Table 1. It reached loss 6.3, a 1.5 gap against the baseline. With several variables changing at once (consumer hardware, mixed backends, residential bandwidth), we could not isolate which fraction of the gap came from the protocol and which from the hardware constraints. The remaining curves move to controlled datacenter setups for that reason.

Curve C moved to datacenter GPUs and we turned the outer learning rate up to 1.0 and let compression keep 40–65% of each delta. The gap was 1.6 and the outer LR turned out to be the

Table 2: Systematic optimization of Distrain against the single-GPU baseline. Gap is plateau loss minus baseline (4.8). All distributed curves train the 125M model on FineWeb-Edu. Curve F is excluded from the gap column because its training loss was corrupted by a data loader bug (see §5.4).

Curve	Hardware	Loss	Gap	Tokens	Key change
A (BL)	1× RTX 4000 Ada	4.8	—	∼35M	Single GPU, batch=4, lr=3e-4, cosine
B	3× Mac (consumer)	6.3	+1.5	46.7M	First distributed run, multiple days wall time
C	3× mixed RTX	6.4	+1.6	∼30M	Outer LR→1.0, top-k 40–65%
D	3× RTX A4000	6.0	+1.2	∼30M	Patience triggers, all nodes contribute
E	3× A40	5.8	+1.0	∼30M	Continuous training (GPU never idles)
F	3× A40	5.96	—	∼17M	Batch=25 + LR scaling, data loader bug (excluded)
G	3× A40	6.4	+1.6	∼17M	Loss-based inner LR, decayed too early
H	3× A40	5.98	+1.2	∼30M	Clean comparison: constant lr=3e-4, persistent data loader

change that mattered. Earlier runs were scaling the merged delta down before applying it, which discards signal we already paid for.

Curve D added the rule that a checkpoint only gets produced after at least half of the active nodes have pushed. Before this, a fast node alone could trigger a checkpoint and the slower nodes' work would race against the version change. With the rule, each merge averages over more contributors, and the average is better, with the gap dropping to 1.2. The 0.5M-token spike to 121.1 in the curve is the first checkpoint, produced from very few contributions before the rule had anything to wait on.

Curve E was about not letting the GPU sit idle. We pulled checkpoint download and shard reload off the training thread, and pushed compression to a separate thread that runs while the next round trains. The gap dropped to 1.0, the lowest we have. But Curve E changed three things together. The number is good and the engineering is real, but it is not the protocol's number alone.

Curve G tested a loss-based inner learning rate that decays as the model converges: $\mathrm{lr} = \mathrm{lr}_{\max} \times \mathrm{clamp}(\mathrm{loss}_{\mathrm{ema}}/\ln V,\ 0.1,\ 1.0)$. Early on it worked: 7.8 at 5M tokens vs Curve H's 8.6. The decay started before the model had finished learning, though, and Curve G ended 0.42 points above H. We went back to constant inner LR.

Curve H is the comparison we wanted from the start. Same batch, same learning rate, same schedule function, same persistent data loader as the single-GPU baseline. Loss 5.98, gap 1.2. This is the number we use as the headline result, not because it is the lowest but because it is the cleanest. Curve E is lower (5.8, gap 1.0) but its gap mixes protocol cost with engineering. Curve H separates them.

Validation. We held out shards from the training assignment and evaluated final checkpoints on them. The validation loss came within 0.4% of the training loss across runs. With each node consuming around 30M tokens out of about 2.2B available to it, repeated exposure was not in play, and training loss is a fair proxy for generalization.

5.3 Batch Size and Token Efficiency

Curve F was an attempt to use the larger batch the A40s could fit. The calibration code chose batch 25 from the available 48 GB of VRAM, and we applied linear LR scaling ($\mathrm{lr} = 3\mathrm{e}{-4} \times 25/4 = 1.875\mathrm{e}{-3}$). The run finished its wall-clock budget faster than batch 4 would have, but made fewer gradient updates per token consumed.

Smaller batches give more gradient updates per token, and more updates mean better per-token

Table 3: Per-token efficiency of small vs large batch on the 125M model. Linear LR scaling adjusts update magnitude but not update frequency, so batch 4 makes more gradient updates per token consumed.

Batch	LR	Tokens/step	Gradient updates per 1M tokens
4	3e-4	2,048	488
25	1.875e-3	12,800	78

convergence. The linear scaling rule compensates for update magnitude but not for update frequency. The loss numbers from Curve F itself are not in Table 2 because the run was also affected by the data loader bug described in §5.4; the batch-size comparison above is over wall-clock and update count and is independent of that bug. All other reported curves use batch 4.

5.4 Negative Results

Loss-based inner LR (Curve G). The adaptive LR $\mathrm{lr} = \mathrm{lr}_{\max} \times \mathrm{clamp}(\mathrm{loss}_{\mathrm{ema}}/\ln V,\ 0.1,\ 1.0)$ gives full learning rate while loss exceeds $\ln V \approx 10.4$, then decays smoothly. Curve G converged faster than Curve H early on (7.8 vs. 8.6 at 5M tokens) but plateaued at 6.4 against H's 5.98, ending 0.42 points higher. The decay started biting before the model had finished learning. Constant $\mathrm{lr} = 3\mathrm{e}{-4}$ is more robust for this protocol.

Data loader state persistence. In earlier runs, before the persistent data loader was added, the loader reset to position 0 every time the node pulled a new checkpoint. Each checkpoint brought a fresh assignment of shards, the loader started those shards from the beginning, and over many checkpoints the model trained on roughly the same prefix of each shard repeatedly. Training loss dropped toward zero while validation loss did not, and the model was memorizing, not generalizing. The metric that exposed this was the gap between training and held-out validation loss; nothing in the per-round logs looked wrong. Curve F was the labeled run that surfaced the bug. The fix was to persist the loader's position across checkpoint changes, which all curves in Table 2 (other than F) use.

Low-rank delta compression. Rank-32 decomposition of training-from-scratch deltas gave 88–98% reconstruction error. Pre-training deltas span many dimensions simultaneously: AdamW [12] on diverse data produces updates that are not concentrated in a small subspace. Low-rank compression works for fine-tuning [6], where the changes to a pretrained model are structurally low-rank, but not for pretraining. Top-k sparsification with error feedback was the only compression method we found that worked in this regime.

5.5 Communication–Quality Tradeoff

The protocol works at retention levels from 1% to 90%, with the upload size and the merge quality moving in opposite directions:

The bandwidth-driven retention selection picks a value that keeps each upload under roughly ten seconds at the node's measured bandwidth. We have not run a controlled study isolating retention from the other variables in Curves B–D, so we cannot give a per-point cost of dropping retention in this paper.

Table 4: Bandwidth-driven retention selection. Each row shows the retention the calibration code picks for a node with the listed upload bandwidth, with the resulting compressed delta size and upload time for a 125M model.

Connection	Upload	Delta Size	Retention	Upload Time
Datacenter	50+ MB/s	∼300 MB (top-k 90%)	90%	∼6s
Fast residential	10 MB/s	∼50 MB (top-k 20%)	20%	∼5s
Slow residential	1 MB/s	∼5 MB (top-k 2%)	2%	∼5s

5.6 Preliminary 1B Scaling

We also ran the protocol on a 1.13B model (2048 hidden, 24 layers, 16 attention heads, 4 KV heads) on three consumer devices (M2 Pro GPU, M2 GPU, Intel i7 CPU) for five days. Loss dropped from 1,669 at random init to 48.8 over 63 checkpoints and roughly 3.5M tokens. The model is still very early in training at this loss level; we report the run to show that the protocol works at 1B, not as a convergence result.

This run predates the persistent data loader and the error buffer persistence fix, and it has no single-GPU baseline, so we keep it out of the systematic table. The implementation handled checkpoints of around 4.5 GB, compression reduced each delta to about 25 MB (175×), and all three devices contributed across the five days despite the 10× throughput gap between fastest and slowest. A controlled 1B comparison with a matched baseline is on our list for the next version of this work.

6 Discussion

The clean comparison at 125M (Curve H) reaches loss 5.98 against the single-GPU baseline of 4.8, a 1.2-point gap. With engineering optimizations that take the GPU off the critical path during checkpoint download and delta upload, Curve E reaches 5.8, a 1.0-point gap, but mixes protocol cost with engineering. Without an experiment that varies one of the contributing factors at a time, we cannot decompose either gap into shares. Three things plausibly contribute: top-k sparsification drops gradient signal that the error feedback buffer only partially recovers across checkpoint boundaries, deltas are weighted-averaged rather than applied sequentially, and merge staleness adds delay between when a node trains and when its contribution affects the model. Sorting out which dominates is the first thing we will measure in v0.2.

Top-k sparsification with error feedback was the only compression method we found that worked. Low-rank decomposition, which is effective for fine-tuning where updates are structurally low-rank [6], gave 88–98% reconstruction error on pretraining deltas; the updates from AdamW on diverse data span too many dimensions for a rank-32 decomposition to capture. Top-k pays an index overhead in exchange for keeping the largest-magnitude updates intact, and in our pipeline that tradeoff was acceptable. We have not run a controlled study comparing structured block sparsity, importance-weighted selection, or quantization variants against unstructured top-k; these are on our list for v0.2.

The protocol's preference for high-retention deltas pulls against the bandwidth volunteer hardware actually has. Datacenter connections can carry ∼300 MB deltas at 90% retention with negligible upload time; residential connections need to drop retention to 1–2% to keep upload under ten seconds. The bandwidth-driven retention selection in §3.3 is the system's way of accommodating both, but it is a per-node choice, not a per-checkpoint quality guarantee. In a deployment with mostly slow nodes the merged checkpoint will see lower retention on average, and the gap will widen.

We have not measured this relationship at scale.

The work has clear limitations. Three nodes is not many. The systematic table reports loss against a single baseline and does not include confidence intervals from multiple seeds. The 1B run is preliminary and predates two relevant bug fixes; we treat it as evidence that the implementation handles the larger sizes.

The current implementation runs a single coordinator with a hybrid in-memory/snapshot state design that would race if multiple coordinators were deployed against the same bucket; the multi-coordinator extension is straightforward in principle and uncoded in practice. We do not provide a formal convergence analysis. The protocol has structural similarities with stale synchronous parallel methods [5], but multi-step weight deltas, error buffer resets at checkpoint boundaries, and discrete-cycle weighted merging put it outside the cases existing theorems cover.

6.1 Future Directions

The next version of this work, v0.2, is about closing the 1.2-point gap and validating beyond three nodes. We are working on better delta compression (structured block sparsity, per-block quantization, importance-weighted selection), staleness handling that recovers signal from late deltas instead of discarding them, and a public volunteer campaign to measure how the gap behaves with diverse hardware and many more nodes. A controlled 1B comparison with a matched single-GPU baseline is the experiment we most want to add.

7 Conclusion

Distrain is a distributed training system that works without synchronization between nodes, runs on heterogeneous hardware from laptops to datacenter GPUs, and ships as a single Rust binary. We characterized it on a 125M transformer across seven distributed curves plus a single-GPU baseline. The clean comparison (Curve H) reaches loss 5.98 against the baseline's 4.8, a 1.2-point gap. With engineering optimizations on top of the protocol, Curve E reaches 5.8, a 1.0-point gap. Top-k sparsification with error feedback was the only compression that worked for pretraining deltas; low-rank decomposition, loss-based inner LR, and resetting the data loader between checkpoints all failed.

This paper does not yet establish how the gap decomposes across its plausible sources. It does not show how the protocol behaves at the node counts a real volunteer deployment would have. It also does not include a controlled comparison at 1B against a single-GPU baseline. The next version of this work will address these shortcomings. Source code: https://github.com/frane/distrain.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, December 2023. URL http://arxiv.org/abs/2305.13245. arXiv:2305.13245 [cs].

[2] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative Inference and Fine-tuning of Large Models, March 2023. URL http://arxiv.org/abs/2209.01188. arXiv:2209.01188 [cs].

[3] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, and Andrew Y Ng. Large Scale Distributed Deep Networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.

[4] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed Low-Communication Training of Language Models, September 2024. URL http://arxiv.org/abs/2311.08105. arXiv:2311.08105 [cs].

[5] Qirong Ho, James Cipar, Henggang Cui, Seunghak Lee, Jin Kyu Kim, Phillip B Gibbons, Garth A Gibson, Greg Ganger, and Eric P Xing. More Effective Distributed ML via a Stale Synchronous Parallel Parameter Server. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, October 2021. URL http://arxiv.org/abs/2106.09685. arXiv:2106.09685 [cs].

[7] Sami Jaghouar, Jack Min Ong, and Johannes Hagemann. OpenDiLoCo: An Open-Source Framework for Globally Distributed Low-Communication Training, July 2024. URL http://arxiv.org/abs/2407.07852. arXiv:2407.07852 [cs].

[8] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian U. Stich, and Martin Jaggi. Error Feedback Fixes SignSGD and other Gradient Compression Schemes, May 2019. URL http://arxiv.org/abs/1901.09847. arXiv:1901.09847 [cs].

[9] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13 (12), 2020. doi: 10.14778/3415478.3415530.

[10] Tao Lin, Sebastian U. Stich, Kumar Kshitij Patel, and Martin Jaggi. Don't Use Large Mini-Batches, Use Local SGD, February 2020. URL http://arxiv.org/abs/1808.07217. arXiv:1808.07217 [cs].

[11] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J. Dally. Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training, June 2020. URL http://arxiv.org/abs/1712.01887. arXiv:1712.01887 [cs].

[12] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, January 2019. URL http://arxiv.org/abs/1711.05101. arXiv:1711.05101 [cs].

[13] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data, April 2017. URL http://arxiv.org/abs/1602.05629. arXiv:1602.05629 [cs].

[14] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale, October 2024. URL http://arxiv.org/abs/2406.17557. arXiv:2406.17557 [cs].

[15] Bowen Peng, Lizhang Chen, Baiyu Su, Jeffrey Quesnelle, Diederik P. Kingma, and Qiang Liu. DeMo: Decoupled Momentum Optimization, November 2024. URL http://arxiv.org/abs/2411.19870. arXiv:2411.19870 [cs].

[16] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, May 2020. URL http://arxiv.org/abs/1910.02054. arXiv:1910.02054 [cs].

[17] Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. Hogwild: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. In *Advances in Neural Information Processing Systems*, volume 24, 2011.

[18] Max Ryabinin and Anton Gusev. Towards Crowdsourced Training of Large Neural Networks using Decentralized Mixture-of-Experts, October 2020. URL http://arxiv.org/abs/2002.04013. arXiv:2002.04013 [cs].

[19] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. SWARM Parallelism: Training Large Models Can Be Surprisingly Communication-Efficient, June 2023. URL http://arxiv.org/abs/2301.11913. arXiv:2301.11913 [cs].

[20] Noam Shazeer. GLU Variants Improve Transformer, February 2020. URL http://arxiv.org/abs/2002.05202. arXiv:2002.05202 [cs].

[21] Nathaniel Simard, Louis Fortier-Dubois, Dilshod Tadjibaev, Guillaume Lagrange, and Burn Framework Contributors. Burn: A flexible and comprehensive deep learning framework in Rust. https://github.com/tracel-ai/burn, 2024. Version 0.14.0.

[22] Sebastian U. Stich. Local SGD Converges Fast and Communicates Little, May 2019. URL http://arxiv.org/abs/1805.09767. arXiv:1805.09767 [math].

[23] Sebastian U. Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. Sparsified SGD with Memory, November 2018. URL http://arxiv.org/abs/1809.07599. arXiv:1809.07599 [cs].

[24] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding, November 2023. URL http://arxiv.org/abs/2104.09864. arXiv:2104.09864 [cs].

[25] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models, February 2023. URL http://arxiv.org/abs/2302.13971. arXiv:2302.13971 [cs].

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. URL http://arxiv.org/abs/1706.03762. arXiv:1706.03762 [cs].

[27] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization, October 2019. URL http://arxiv.org/abs/1910.07467. arXiv:1910.07467 [cs].

[28] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania,

Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel, September 2023. URL http://arxiv.org/abs/2304.11277. arXiv:2304.11277 [cs].